PHANTOM CONSENSUS

TEAM EXECUTION BLUEPRINT — 3-Person Hackathon War Plan

STRATEGIC OVERVIEW

P1 — ARCHITECT

Infrastructure, Pipeline, Data Layer, CI/CD, Config

P2 — CORE LOGIC

Scoring, Strategy, Filters, Alliance Detection, Consensus

P3 — QA + OUTPUT

All 18 Scenario Tests, Edge Cases, Output, Dashboard

PARALLEL EXECUTION TIMELINE

PHASE	P1 — ARCHITECT	P2 — CORE LOGIC	P3 — QA + OUTPUT
0 – 30 min	Repo setup, pyproject, Makefile, config.py, models/	Study problem, draft algorithm pseudocode, formula whiteboard	Write conftest.py, all 18 scenario data fixtures
30 – 75 min	Ingestion layer: JSON + CSV parsers, loader.py	relationship_scorer.py + objection_scorer.py	Unit tests for sanitization modules (mock-driven)
75 – 120 min	Sanitization: id_normalizer, type_coercion, dedup, ref_validator	viability_scorer, trojan_filter, poison_filter	Scenario tests: trojan, poison_pill, false_friend, supporter_coherence
120 – 165 min	CI/CD: GitHub Actions yml, coverage gate, structlog setup	alliance_detector, infiltrator_scan, consensus_builder	Scenario tests: cascading_betrayal, infiltrator, scale, dirty_csv
165 – 210 min	Profiler, Rich dashboard, --query CLI flag, APPROACH.md	Fix threshold calibration (data-driven), Floyd-Warshall cascading	Adversarial fuzz fixtures, Hypothesis property tests, full run
210 – 240 min	Final integration, smoke test, submission packaging	Beam search / combo enumeration for greedy fix, final review	18/18 scenario pass check, coverage gate 90%+, final output verify

■ KEY PARALLELISM PRINCIPLE: P1 pushes skeleton stubs (empty functions with correct signatures) by T+30min so P2 and P3 can import and work independently without merge conflicts. P3 writes tests against interfaces before P2 finishes implementations — TDD.

ROLE DEEP-DIVES

P1 THE ARCHITECT — Infrastructure, Data Pipeline & DevOps

OWNS (Issues)	1, 2, 3, 4, 5 (parsers), 6, 7, 8, 9 (sanitization), config.py, CI/CD, logging, profiler, dashboard, APPROACH.md
DOES NOT TOUCH	scoring/, strategy/ — those are P2 territory. tests/scenarios/ — P3 territory.
FIRST COMMIT	T+20min: repo skeleton, pyproject.toml, Makefile, models/ with Pydantic stubs, config.py with ALL thresholds
CRITICAL UNLOCK	T+30min: Push loader.py with stub returns so P2 can import data structures immediately
Skill Profile	Strong in file I/O, data wrangling, pandas, Pydantic, DevOps. Does not need to know the game theory.

P2 THE STRATEGIST — Core Algorithms, Scoring & Decision Logic

OWNS (Issues)	10, 11, 12, 13 (scoring), 14, 15, 16, 17 (strategy) — relationship_scorer, objection_scorer, viability_scorer, trojan_filt
DOES NOT TOUCH	ingestion/, sanitization/ — P1's territory. tests/scenarios/ — P3's territory.
STARTS AT	T+30min once P1 pushes models/ and config.py. Work from stubs in scoring/ and strategy/ immediately.
CRITICAL FIXES	Data-driven threshold calibration (NOT hardcoded 0.7). Floyd-Warshall cascading betrayal. Beam search for propos
Skill Profile	Strong in NumPy, graph algorithms, NetworkX, optimization, discrete math. Needs to understand the 18 hidden test

P3 THE VALIDATOR — Testing, QA, Output & Submission

OWNS (Issues)	18 (formatter.py), 19 (edge cases), 20 (profiling), ALL tests/ — unit, integration, all 18 scenario tests, adversarial fuz
DOES NOT TOUCH	src/ implementation files — read them, import from them, but don't edit them.
STARTS AT	T+0min immediately — write conftest.py and all 18 JSON scenario fixtures while P1 sets up repo. Tests will fail until
CRITICAL PATH	Get 18/18 scenario tests to pass. This alone is the difference between 1st and 3rd place. Also owns the final output
Skill Profile	Strong in pytest, test design, JSON schema validation, edge case thinking. Needs to understand what each hidden t

■ MERGE CONFLICT PREVENTION RULES

Rule	Detail
P1 owns src/ingestion/ + src/sanitization/ + src/models/ + src/utils/ + root config	P2 and P3 IMPORT from these but never edit them. If a bug is found, P2/P3 files a GitHub issue and P1 fixes.
P2 owns src/scoring/ + src/strategy/	P1 and P3 never touch these files. P3 writes tests that import from them.
P3 owns tests/ + src/output/ + dashboard/	P1 and P2 never touch tests/. formatter.py is P3 territory.
Shared: config.py changes	ONLY P1 edits config.py. P2 asks P1 via chat to change a threshold constant — P1 PRs it in 2 min.

PHASE 0 · T+0 → T+30 MIN

IMMEDIATE PARALLEL BLAST — ALL THREE START NOW

STEP 1 — P1: Git Repo + Project Skeleton — P1

- Create GitHub repo, push initial commit with .gitignore, README stub
- Create pyproject.toml with all dependencies listed
- Create Makefile with targets: run, test, lint, format, dashboard, clean
- Create src/__init__.py and all subdirectory __init__.py files
- Create src/config.py with ALL constants (thresholds commented with rationale)

■ ANTIGRAVITY PROMPT:

```
You are setting up a production Python 3.12 hackathon project called phantom-consensus.
Create pyproject.toml with: numpy>=1.26, pandas>=2.0, networkx>=3.2, pydantic>=2.5,
structlog>=23.0, rich>=13.0, typer>=0.9, pytest>=7.4, pytest-cov>=4.1, hypothesis>=6.90.
Also create src/config.py with these externalized constants with docstring rationale:
TROJAN_BETRAYAL_THRESHOLD, TROJAN_INFLUENCE_THRESHOLD, ALLIANCE_MIN_SCORE,
ALLIANCE_MIN_MEMBERS, MIN_VIABILITY, MAX_PROPOSALS, MAX_SUPPORTERS,
INFILTRATOR_THRESHOLD, NULL_INFLUENCE_STRATEGY. Make thresholds data-adaptive hooks.
```

■ MANUAL STEP:

→ mkdir -p src/{models,ingestion,sanitization,scoring,strategy,output,utils} tests/{unit,integration,scenarios} data output dashboard docs

■ TEST:

```
python -c 'from src.config import TROJAN_BETRAYAL_THRESHOLD; print(TROJAN_BETRAYAL_THRESHOLD)' # must not crash
```

STEP 2 — P1: Pydantic Models — The Shared Contract — P1

- Create src/models/representative.py — Representative + ValidatedRep with influence coercion
- Create src/models/proposal.py — Proposal with priority field
- Create src/models/objection.py — Objection with severity + rep_id
- Create src/models/relationship.py — Relationship + RelationshipScore dataclass
- These models are the SHARED CONTRACT — P2 and P3 import from here
- Push to main immediately — this is the unlock for P2 and P3

■ ANTIGRAVITY PROMPT:

```
Create Pydantic v2 models for a political consensus engine. Representative has: id (str),
name (str), faction (str), influence (Optional[int]). Add a field_validator for influence
that casts strings to int, clamps to [0,100], returns None for unparseable values.
Relationship has: from_rep, to_rep (str), trust (float 0-100), rivalry (float), betrayal_prob
(float 0-1).
RelationshipScore is a dataclass with from_rep, to_rep, score (float).
Use model_config = ConfigDict(strict=False) for flexibility in parsing.
```

■ TEST:

```
python -c "from src.models.representative import Representative;
r=Representative(id='rep_001',name='x',faction='A',influence='85'); assert r.influence==85"
python -c "from src.models.representative import Representative;
r=Representative(id='x',name='x',faction='A',influence=150); assert r.influence==100"
```

STEP 3 — P2: Algorithm Whiteboard + Pseudocode — P2

- DO NOT WRITE CODE YET — spend first 30 min on algorithm design
- Write pseudocode for relationship_scorer: trust × (1 - betrayal_prob) / 100
- Write pseudocode for alliance_detector: bidirectional min-score graph + connected components
- Map all 18 hidden test scenarios to the specific module that catches each one
- Identify the cascading betrayal fix: Floyd-Warshall or damped BFS depth 3
- Identify the threshold calibration fix: mean + 0.5*std from input data
- Write this in docs/ALGORITHM_NOTES.md — push to repo

■ MANUAL STEP:

→ Open docs/ALGORITHM_NOTES.md and write out formulas for all 6 scoring functions

→ Draw the 14-stage pipeline DAG on paper — identify which stages P2 owns (stages 6-13)

→ Confirm with P1 what the RelationshipMatrix dataclass looks like before writing scorer

STEP 4	P3: conftest.py + All 18 Scenario Data Fixtures	P3

- Create tests/conftest.py with build_scenario() factory and run_engine() fixture
- Create tests/scenarios/ directory with one file per hidden test scenario
- Each scenario file has crafted JSON data that WILL trigger that specific trap
- Tests will FAIL until P2 finishes — that is expected and correct (TDD)
- This is the most important P3 task — spend 25 minutes here

■ ANTIGRAVITY PROMPT:

```
Create a pytest conftest.py for a consensus engine. build_scenario(reps, proposals,
objections, relations) returns a dict of loaded data. run_engine(scenario) calls the
full pipeline and returns the final_agreement.json content as a dict.
Create scenario fixture for 'Trojan Horse': rep_001 has influence=95, betrayal_prob=0.95.
rep_002 is safe with influence=60, betrayal_prob=0.05. One proposal sponsored by rep_002.
Expected: rep_001 NOT in supporting_reps, rep_002 IS in supporting_reps.
Create scenario for 'False Friend': rep_A trusts rep_B score=0.85, rep_B trusts rep_A
score=0.12.
Expected: NO alliance between them.
```

■ TEST:

```
pytest tests/scenarios/ --collect-only # must collect 18 test files without import errors
pytest tests/conftest.py # conftest itself must be importable
```

PHASE 1 · T+30 → T+75 MIN

DATA INGESTION LAYER — P1 BUILDS, P2+P3 USE STUBS

STEP 5 — P1: JSON Parser — reps, proposals, objections — P1

- Create src/ingestion/json_parser.py
- load_representatives(path) → list[dict]
- load_proposals(path) → list[dict]
- load_objections(path) → list[dict]
- Each function: open file, json.load, wrap in try/except, return empty list on failure
- Log file load success/failure with structlog

■ ANTIGRAVITY PROMPT:

```
Create src/ingestion/json_parser.py for a political consensus engine. Each function
takes a Path, loads JSON, catches json.JSONDecodeError and FileNotFoundError,
logs the error with structlog (event='file_load_failed', path=str(path), error=str(e)),
and returns an empty list on failure. Functions: load_representatives, load_proposals,
load_objections. Add type hints with list[dict]. Each success also logs event='file_loaded'
with record_count=len(data).
```

■ TEST:

```
pytest tests/unit/test_json_parser.py # test with valid + invalid + missing files
python -c "from src.ingestion.json_parser import load_representatives;
print(load_representatives('data/representatives.json'))"
```

STEP 6 — P1: CSV Parser — relations.csv with row-level fault isolation — P1

- Create src/ingestion/csv_parser.py
- load_relationships(path) → list[dict] with ROW-LEVEL try/except
- Bad rows go to quarantine log — they do NOT crash the parser
- Use pandas read_csv with dtype=str to avoid silent type coercion
- Log quarantined rows with row number and error
- This is the 'Dirty CSV Parsing' hidden test — row-level isolation is the key

■ ANTIGRAVITY PROMPT:

```
Create src/ingestion/csv_parser.py. Use pandas to read relations.csv where columns are:
from_rep, to_rep, trust, rivalry, betrayal_prob. Use dtype=str on load.
Then iterate rows with iterrows(). In each row, wrap parsing in try/except:
cast trust/rivalry to float, betrayal_prob to float clamped [0,1].
On failure, log with structlog event='row_quarantined', row_index=i, error=str(e).
Return only successfully parsed rows as list[dict]. Never raise on bad rows.
```

■ TEST:

```
pytest tests/unit/test_csv_parser.py -v # include a fixture CSV with 1 bad row
# Confirm: bad row logged but other rows returned correctly
```

STEP 7 — P1: Loader Orchestrator — P1

- Create src/ingestion/loader.py — the single entry point that calls all parsers
- load_all(data_dir: Path) → RawData (TypedDict or dataclass with all 4 lists)
- This is what the main entry point calls — one function, clean interface
- Push immediately — P2 can now write scoring code that uses RawData

■ **ANTIGRAVITY PROMPT:**

```
Create src/ingestion/loader.py. Define RawData as a TypedDict with keys:
representatives: list[dict], proposals: list[dict], objections: list[dict],
relationships: list[dict]. load_all(data_dir: Path) -> RawData calls all four parsers
and returns the combined result. Log event='all_data_loaded' with counts for each.
```

■ **TEST:**

```
python -c "from src.ingestion.loader import load_all; from pathlib import Path;
d=load_all(Path('data')); print(d.keys())"
```

STEP 8 | **P2: Relationship Scorer (starts at T+30 after models push)** | **P2**

- Create src/scoring/relationship_scorer.py
- build_score_matrix(relationships, rep_ids) → RelationshipMatrix (numpy backed)
- Formula: score[i][j] = trust × (1 - betrayal_prob) / 100
- get_bidirectional_score(matrix, a, b) → min(score[a→b], score[b→a])
- Use numpy zeros matrix — this is the scale correctness fix
- Handle missing pairs: default to 0.0 score (not an error)

■ **ANTIGRAVITY PROMPT:**

```
Create src/scoring/relationship_scorer.py with numpy. RelationshipMatrix is a dataclass
with rep_ids: list[str] and scores: np.ndarray of shape (n,n) float64.
build_score_matrix(relationships: list[dict], rep_ids: list[str]) -> RelationshipMatrix:
Build index map, initialize np.zeros((n,n)). For each relationship dict, compute
score = trust * (1.0 - betrayal_prob) / 100.0. Clamp to [0,1].
get_bidirectional_score returns min of both directions. If either rep not in matrix, return
0.0.
```

■ **TEST:**

```
pytest tests/unit/test_relationship_scorer.py -v
# Test: trust=100, betrayal=0.0 → score=1.0
# Test: trust=100, betrayal=1.0 → score=0.0
# Test: asymmetric → bidirectional returns the minimum
```

STEP 9 | **P2: Objection Weight Calculator** | **P2**

- Create src/scoring/objection_scorer.py
- compute_objection_weights(objections, reps_by_id) → dict[prop_id, float]
- Formula: Σ(severity_i × influence_i) for all objectors of that proposal
- If influence is None: skip that objector from sum (do NOT crash)
- Normalize to [0,1] by dividing by max possible (100 × 100 = 10000)
- Return 0.0 for proposals with no objections

■ **ANTIGRAVITY PROMPT:**

```
Create src/scoring/objection_scorer.py. compute_objection_weights takes
objections: list[dict] and reps_by_id: dict[str, Representative].
Group objections by proposal_id. For each proposal, sum severity * influence
for each objector, skipping None influence. Normalize by dividing by 10000.
Return dict mapping proposal_id -> float [0,1]. Missing proposals default to 0.0.
```

■ **TEST:**

```
pytest tests/unit/test_objection_scorer.py -v
# Test: no objections → 0.0
```

```
# Test: null influence objector → excluded from sum, no crash
```

STEP 10	P3: Unit Tests for Sanitization (mock-driven, no real data needed)	P3

- Create tests/unit/test_id_normalizer.py — test all ID formats
- Create tests/unit/test_type_coercion.py — test influence casting edge cases
- Create tests/unit/test_deduplicator.py — test duplicate detection and rule
- Create tests/unit/test_ref_validator.py — test ghost reference detection
- Use @pytest.mark.parametrize for bulk test cases
- These tests run INDEPENDENTLY of P2 work — zero dependency

■ ANTIGRAVITY PROMPT:

```
Create tests/unit/test_id_normalizer.py. Test normalize_rep_id() with these inputs:
'REP_001' → 'rep_001', 'rep_1' → 'rep_001', ' Rep-002 ' → 'rep_002', 'REP001' → 'rep_001'.
Also test normalize_prop_id: 'PROP_01' → 'prop_01', 'Prop 002' → 'prop_002'.
Use @pytest.mark.parametrize with a list of (input, expected) tuples.
Import from src.sanitization.id_normalizer (these will fail until P1 creates the file — that is fine).
```

■ TEST:

```
pytest tests/unit/ --tb=short 2>&1 | tail -20 # expect import errors until P1 pushes, not test logic errors
```

PHASE 2 · T+75 → T+120 MIN

SANITIZATION LAYER — PARALLEL: P1 IMPLEMENTS, P2+P3 BUILD ON MODELS

STEP 11 — P1: ID Normalizer (Issue 6) — P1

- Create src/sanitization/id_normalizer.py
- normalize_rep_id(raw) → canonical 'rep_001' format (strip, lower, zfill to 3 digits)
- Handle: 'REP_001', 'rep_1', 'Rep-002', ' REP_1 ', 'REP001' — all → 'rep_001' format
- normalize_prop_id(raw) → lowercase with underscores
- Run normalizer on ALL datasets before any cross-referencing
- This is the 'ID Normalization' hidden test — case sensitivity kills naive solutions

■ ANTIGRAVITY PROMPT:

```
Create src/sanitization/id_normalizer.py. Use regex: r'^(?:rep|REP|Rep)[_\-]?(\d+)$'
normalize_rep_id(raw: Any) -> str | None: strip, lowercase, match regex,
return f'rep_{match.group(1).zfill(3)}'. If no match, return cleaned string as-is.
Return None if input is not a string. normalize_all_datasets(raw_data: RawData) -> RawData:
applies normalization to id fields across all four datasets in-place, returns normalized data.
```

■ TEST:

```
pytest tests/unit/test_id_normalizer.py -v # should now all pass
```

STEP 12 — P1: Type Coercion + Null Handler (Issue 7) — P1

- Create src/sanitization/type_coercion.py
- coerce_representative(raw_dict) → Representative | None
- Influence: cast string→int, clamp [0,100], None→quarantine but don't crash
- Records that fail coercion go to quarantine log, not thrown away silently
- Implement median imputation for None influence (collect valid values, compute median)
- This is the 'Null Influence Handling' hidden test

■ ANTIGRAVITY PROMPT:

```
Create src/sanitization/type_coercion.py. coerce_influence(v) -> int | None:
try int(float(str(v).strip())), except return None. Then clamp: max(0, min(100, parsed)).
coerce_all_representatives(reps: list[dict]) -> tuple[list[Representative], list[dict]]:
Returns (valid_reps, quarantined). For None influence after coercion, impute with
median of valid influences. Log quarantined with structlog event='record_quarantined'.
```

■ TEST:

```
pytest tests/unit/test_type_coercion.py -v # all edge cases should pass
```

STEP 13 — P1: Deduplicator + FK Validator (Issues 8 & 9) — P1

- Create src/sanitization/deduplicator.py — deduplicate proposals by id, keep highest priority
- Create src/sanitization/ref_validator.py — validate all FK references across datasets
- Ghost sponsors: proposal references rep_id not in representatives → quarantine proposal
- Ghost objectors: objection references rep_id not in representatives → quarantine objection
- All quarantined records emitted to quarantine log (judges see this as engineering rigor)

■ ANTIGRAVITY PROMPT:

```
Create src/sanitization/deduplicator.py. deduplicate_proposals(proposals: list[Proposal])
-> list[Proposal]: group by id, keep the one with highest priority. Log duplicates with
```

```
structlog event='duplicate_removed', kept_priority, removed_count.
Create src/sanitization/ref_validator.py. validate_references(reps, proposals, objections)
-> tuple[valid_proposals, valid_objections]: remove proposals whose sponsor_id not in
rep_id_set, remove objections whose rep_id or proposal_id not in valid sets. Log each removal.
```

■ **TEST:**

```
pytest tests/unit/test_deduplicator.py tests/unit/test_ref_validator.py -v
# Test ghost sponsor: proposal with unknown sponsor should be quarantined
# Test dedup: two prop_001 entries → keep higher priority one
```

STEP 14 | **P2: Viability Scorer (Issue 13)** | **P2**

- Create src/scoring/viability_scorer.py
- compute_viability(proposal, obj_weights) → float [0,1]
- Formula: priority × (1 - controversy) where controversy = min(1.0, obj_weight / threshold)
- controversy_threshold = 0.6 × max_obj_weight in dataset (relative, not absolute)
- This makes the poison pill detection adaptive to the dataset distribution
- This is the 'Priority vs Objection Weight' and 'Poison Pill' hidden test fix

■ **ANTIGRAVITY PROMPT:**

```
Create src/scoring/viability_scorer.py. compute_viability_scores(
proposals: list[Proposal], obj_weights: dict[str, float]) -> dict[str, float]:
First compute controversy_threshold = 0.6 * max(obj_weights.values(), default=1.0).
For each proposal: controversy = min(1.0, obj_weights.get(p.id, 0.0) / controversy_threshold).
viability = (p.priority / 10.0) * (1.0 - controversy). Clamp to [0,1].
Return dict proposal_id -> viability_score.
```

■ **TEST:**

```
pytest tests/unit/test_viability_scorer.py -v
# Test: zero objection weight → viability = priority/10
# Test: max obj weight → viability near 0 (poison pill rejected)
```

STEP 15 | **P3: Scenario Tests — Trojan, Poison Pill, False Friend, Supporter Coherence** | **P3**

- Complete tests/scenarios/test_trojan_horse.py — 2 assertions
- Complete tests/scenarios/test_poison_pill.py — high priority + heavy objection → rejected
- Complete tests/scenarios/test_false_friend.py — asymmetric trust → no alliance
- Complete tests/scenarios/test_supporter_coherence.py — objector never in supporting_reps
- Each file: build_scenario() with crafted data, run_engine(), assert expected output
- These 4 tests cover ~24 points of the hidden test suite

■ **ANTIGRAVITY PROMPT:**

```
Create tests/scenarios/test_poison_pill.py. Build scenario: prop_001 has priority=10
(maximum), but 3 influential reps (influence=90,85,80) each object with severity=10.
prop_002 has priority=6, no objections. Expected: prop_002 selected, prop_001 rejected.
Create tests/scenarios/test_supporter_coherence.py: rep_003 objects to prop_001.
prop_001 is selected. Expected: rep_003 NOT in supporting_reps even if rep_003 has
high influence. This tests explicit objector exclusion from supporter set.
```

■ **TEST:**

```
pytest tests/scenarios/test_trojan_horse.py tests/scenarios/test_poison_pill.py -v
pytest tests/scenarios/test_false_friend.py tests/scenarios/test_supporter_coherence.py -v
```

PHASE 3 · T+120 → T+165 MIN

STRATEGY LAYER — P2 CORE LOGIC, P1 CI/CD, P3 MORE SCENARIOS

STEP 16 — P2: Trojan Filter (Issue 12) — P2

- Create src/strategy/trojan_filter.py
- is_trojan_horse(rep, matrix) → bool
- Condition: max_outgoing_betrayal > TROJAN_BETRAYAL_THRESHOLD AND influence > TROJAN_INFLUENCE_THRESHOLD
- Compute max betrayal by inverting the score matrix: betrayal = 1 - score (normalized)
- filter_trojans(reps, matrix) → list[Representative] (safe reps only)
- Log each exclusion with event='trojan_detected', rep_id, influence, max_betrayal

■ ANTIGRAVITY PROMPT:

```
Create src/strategy/trojan_filter.py. is_trojan_horse(rep: Representative,
matrix: RelationshipMatrix) -> bool: get rep's row index in matrix.
Extract raw betrayal prob from the original relationships (pass as extra arg or
store in matrix). Flag if betrayal_prob > TROJAN_BETRAYAL_THRESHOLD (from config)
AND rep.influence > TROJAN_INFLUENCE_THRESHOLD. filter_trojans returns reps
where is_trojan_horse is False. Log exclusions with structlog.
```

■ TEST:

```
pytest tests/scenarios/test_trojan_horse.py -v # must pass now that impl exists
```

STEP 17 — P2: Alliance Detector with Asymmetric Trust Fix (Issues 14 & 15) — P2

- Create src/strategy/alliance_detector.py
- detect_alliances(matrix, safe_reps) → list[list[str]]
- Build NetworkX Graph — edge exists ONLY if BOTH directions ≥ ALLIANCE_MIN_SCORE
- This is the asymmetric trust (False Friend) fix — directional validation
- Connected components with ≥2 members = genuine alliances
- Return sorted lists for deterministic output
- DATA-DRIVEN THRESHOLD: compute ALLIANCE_MIN_SCORE = mean(all_scores) + 0.5 × std(all_scores) at runtime

■ ANTIGRAVITY PROMPT:

```
Create src/strategy/alliance_detector.py using networkx. detect_alliances(
matrix: RelationshipMatrix, safe_reps: list[str]) -> list[list[str]]:
First compute adaptive threshold: scores_flat = matrix.scores[matrix.scores > 0].flatten()
adaptive_threshold = float(np.mean(scores_flat) + 0.5 * np.std(scores_flat)) if len > 0 else
ALLIANCE_MIN_SCORE.
Build nx.Graph(), add edge only where score_ab >= adaptive_threshold AND score_ba >=
adaptive_threshold.
Return sorted([sorted(list(c)) for c in nx.connected_components(G) if len(c) >= 2]).
```

■ TEST:

```
pytest tests/scenarios/test_false_friend.py -v # must pass
pytest tests/scenarios/test_clear_alliance.py -v
```

STEP 18 — P2: Faction Infiltrator Scanner (Issue 16) — P2

- Create src/strategy/infiltrator_scan.py
- scan_infiltrators(reps, matrix) → set[str] of flagged rep_ids

- For each faction: compute avg intra-faction betrayal for each member
- If avg betrayal against own faction > INFILTRATOR_THRESHOLD → flagged
- Flagged reps excluded from alliance candidacy within that faction
- This is the 'Faction Infiltrator' hidden test — shared faction label ≠ safety

■ ANTIGRAVITY PROMPT:

```
Create src/strategy/infiltrator_scan.py. scan_infiltrators(
reps: list[Representative], matrix: RelationshipMatrix) -> set[str]:
Group reps by faction. For each faction and each member, compute avg score
against all other faction members using matrix.scores. If avg_score <
(1 - INFILTRATOR_THRESHOLD), flag the member. Log event='infiltrator_detected'.
Return set of flagged rep_ids.
```

■ TEST:

```
pytest tests/scenarios/test_faction_infiltrator.py -v
```

STEP 19 — P2: Cascading Betrayal Fix — Floyd-Warshall (CRITICAL RISK FIX) — P2

- CRITICAL: Original architecture uses BFS depth-2 — this misses 3-hop chains
- Fix: implement transitive risk using Floyd-Warshall relaxation on the score matrix
- transitive_risk_matrix = compute_transitive_risk(matrix) before trojan filter runs
- risk(A→C) through B = min(risk(A→B), risk(B→C)) — weakest link in chain
- Run 3 iterations of relaxation (covers depth-3 chains at n=50)
- This fixes the 'Cascading Betrayal' hidden test

■ ANTIGRAVITY PROMPT:

```
Add compute_transitive_risk(matrix: RelationshipMatrix) -> RelationshipMatrix to
src/scoring/relationship_scorer.py. Convert scores to risk: risk[i][j] = 1 - score[i][j].
Run Floyd-Warshall: for k in range(n): for i: for j: risk[i][j] = min(risk[i][j],
risk[i][k]+risk[k][j]).
Clamp all values to [0,1]. Convert back: transitive_score[i][j] = 1 - risk[i][j].
Return new RelationshipMatrix with transitive scores. Use this matrix in trojan_filter and
alliance_detector.
```

■ TEST:

```
pytest tests/scenarios/test_cascading_betrayal.py -v # 3-hop chain must be caught
# Manual: rep_A trusts rep_B (score 0.9), rep_B trusts rep_C (score 0.9),
# rep_C has betrayal_prob=0.98. rep_A should be flagged as risky via transitivity.
```

STEP 20 — P1: CI/CD GitHub Actions + Structlog Config (Issue 11) — P1

- Create .github/workflows/ci.yml — ruff lint, mypy --strict, pytest with 90% coverage gate
- Create src/utils/logger.py — structlog config in JSON mode for production
- Create src/utils/profiler.py — cProfile wrapper that logs per-stage timing
- Push CI — it will show red until all tests pass, that is expected
- Verify ruff + mypy pass on your own modules before pushing

■ ANTIGRAVITY PROMPT:

```
Create .github/workflows/ci.yml for Python 3.12. Steps: checkout, setup-python 3.12,
pip install -r requirements.txt, ruff check src/ tests/, mypy src/ --strict,
pytest tests/ --cov=src --cov-fail-under=90 -v, python consensus_engine.py (smoke test),
pytest tests/scenarios/ -v. Create src/utils/logger.py using structlog.configure() with
ProcessorFormatter and JSONRenderer for production, ConsoleRenderer for development.
Check LOG_FORMAT env var: 'json' → JSONRenderer, else ConsoleRenderer.
```

■ **TEST:**

```
python -c 'from src.utils.logger import get_logger; log=get_logger(); log.info("test", event="ci_check")'
```

STEP 21	P3: Scenario Tests — Cascading Betrayal, Infiltrator, Scale, Dirty CSV	P3

- Complete tests/scenarios/test_cascading_betrayal.py — 3-hop betrayal chain
- Complete tests/scenarios/test_faction_infiltrator.py — intra-faction spy
- Complete tests/scenarios/test_scale_correctness.py — 50 reps, 30 proposals, verify correct output
- Complete tests/scenarios/test_dirty_csv.py — CSV with 1 malformed row, rest processed
- Complete tests/scenarios/test_id_normalization.py — mixed case IDs cross-file
- Each test: build crafted scenario, run full engine, assert specific expected output

■ **ANTIGRAVITY PROMPT:**

```
Create tests/scenarios/test_cascading_betrayal.py. Build scenario: rep_A→rep_B trust=90, rep_B→rep_C trust=90, rep_C has betrayal_prob=0.98 outgoing. rep_A and rep_B should NOT be in alliances with rep_C. rep_C may also be flagged as trojan via transitivity. Create tests/scenarios/test_dirty_csv.py: 5 valid relation rows + 1 row with betrayal_prob='INVALID'. Expected: engine runs successfully, 5 valid relationships processed, 1 quarantined. Check that final_agreement is still populated correctly.
```

■ **TEST:**

```
pytest tests/scenarios/test_cascading_betrayal.py tests/scenarios/test_dirty_csv.py -v
pytest tests/scenarios/test_scale_correctness.py -v
```

PHASE 4 · T+165 → T+210 MIN

INTEGRATION, CRITICAL FIXES & HARDENING

STEP 22 — P2: Consensus Builder + Greedy Fix (Issue 17) — P2

- Create src/strategy/consensus_builder.py
- build_consensus(proposals, safe_reps, obj_weights, viability_scores, alliances) → dict
- GREEDY FIX: for p ≤ 30 proposals, enumerate top-k combinations using itertools.combinations
- Score each combination: sum of viability scores
- Select combination where the resulting supporter set is largest + most diverse factions
- This fixes the 'Alliance Hack / Disruptor' hidden test
- Enforce: objectors to selected proposals are EXCLUDED from supporting_reps

■ ANTIGRAVITY PROMPT:

```
Create src/strategy/consensus_builder.py. build_consensus takes proposals, safe_reps,
obj_weights, viability_scores, objections, alliances. If len(proposals) <= 30:
use itertools.combinations(viable_proposals, min(3, len)) to enumerate top combos.
Score each combo by sum(viability_scores[p.id] for p in combo). Pick best combo.
supporters = [r for r in safe_reps if r.id not in objectors_to_selected_proposals].
Sort supporters by influence descending. Return {'proposals': ids, 'supporting_reps': ids}.
```

■ TEST:

```
pytest tests/scenarios/test_alliance_hack.py -v
pytest tests/scenarios/test_minimum_viable.py -v # 1 rep, 1 proposal edge case
```

STEP 23 — P2: Data-Driven Threshold Calibration (HIGH RISK FIX) — P2

- THIS IS THE MOST IMPORTANT FIX — hardcoded 0.7/0.5 thresholds will misclassify
- Add calibrate_thresholds(matrix, proposals, obj_weights) → dict to src/scoring/
- TROJAN threshold = mean(all betrayal_probs) + 1.0 × std(betrayal_probs)
- ALLIANCE threshold = mean(all scores > 0) + 0.5 × std(scores > 0)
- VIABILITY threshold = mean(viability_scores) — 0.5 × std(viability_scores)
- These replace the hardcoded constants AT RUNTIME — config.py values become fallbacks
- This is the difference between 16/18 and 18/18 on hidden tests

■ ANTIGRAVITY PROMPT:

```
Create src/scoring/threshold_calibrator.py. calibrate_thresholds(
relationships: list[dict], matrix: RelationshipMatrix, obj_weights: dict) -> dict:
Extract all betrayal_prob values. trojan_betrayal = mean + 1.0*std (clamp 0.6-0.95).
Extract non-zero scores from matrix. alliance_min = mean + 0.5*std (clamp 0.3-0.8).
Extract obj_weight values. viability_min = max(0.05, mean - 0.5*std).
Return {'trojan_betrayal': float, 'alliance_min': float, 'viability_min': float}.
If insufficient data (< 3 values), return config defaults.
```

■ TEST:

```
python -c 'from src.scoring.threshold_calibrator import calibrate_thresholds; print("calibrator
importable")'
# Integration: run full pipeline on sample data, print calibrated thresholds to verify they are
sane
```

STEP 24 — P1: Main Entry Point + --query CLI Flag — P1

- Create consensus_engine.py — the CLI entry point using Typer
- Main command: run pipeline end-to-end, write output/final_agreement.json
- Flags: --verbose (structlog ConsoleRenderer), --profile (cProfile output), --dry-run
- --query rep_007: print full decision trace for that entity (trojan? alliance? excluded because?)
- This is the 'Dashboard Is Cosmetic' fix — judges can now probe any entity
- Smoke test: python consensus_engine.py --dry-run must exit 0

■ ANTIGRAVITY PROMPT:

```
Create consensus_engine.py using typer. Main command app() runs the 14-stage pipeline.
Add --query option: if provided, after pipeline run, print a decision trace for that rep_id:
show their influence, faction, trojan status, alliances, and whether they appear in
supporting_reps.
Format as a Rich panel with colored fields. Add --profile flag that wraps pipeline in
cProfile and prints top 10 slowest calls. Ensure all exceptions are caught at top level
with structlog error logging and sys.exit(1).
```

■ MANUAL STEP:

→ python consensus_engine.py --dry-run → must print 'Pipeline complete' and exit 0

→ python consensus_engine.py --query rep_001 → must print decision trace for rep_001

■ TEST:

```
python consensus_engine.py && cat output/final_agreement.json | python -m json.tool
```

STEP 25	P1: Rich Terminal Dashboard (Issue 20 polish)	P1

- Create dashboard/visualize.py using Rich
- Show: data loading progress bar, quarantine counts, detected trojans, detected poison pills
- Show: alliance graph as a text table, final agreement summary
- Add --html flag that exports dashboard as standalone HTML for submission screenshots
- This satisfies 'Dashboard or Visualization' submission requirement

■ ANTIGRAVITY PROMPT:

```
Create dashboard/visualize.py using Rich library. Build a Rich Layout with panels:
Panel 1: 'Data Pipeline' with a progress table showing each stage and record counts.
Panel 2: 'Threats Detected' with a table listing trojans and poison pills.
Panel 3: 'Alliances' showing each alliance as a comma-separated list with member count.
Panel 4: 'Final Agreement' with selected proposals and supporting reps count.
Load data from output/final_agreement.json and structured logs. Add --html flag
that uses rich.console.Console(record=True) and exports with console.export_html().
```

STEP 26	P3: Adversarial Fuzz Fixtures + Hypothesis Property Tests	P3

- Write 5 adversarial scenarios that probe the nastiest edge cases:
- 1. Circular trust chains: A trusts B, B trusts C, C trusts A at 0.9 — no infinite loop
- 2. All reps in one faction — infiltrator scan handles single-faction correctly
- 3. All reps are rivals (all scores < threshold) — output: alliances: []
- 4. Proposals sponsored by each other's objectors — FK validation handles this
- 5. Single representative, single proposal — minimum viable case
- Write Hypothesis tests: output rep_ids always subset of input rep_ids, no duplicate reps in output

■ ANTIGRAVITY PROMPT:

```
Create tests/scenarios/test_adversarial.py with 5 adversarial fixtures.
Fixture 'circular_trust': A→B trust=90 betrayal=0.05, B→C trust=90 betrayal=0.05,
```

C→A trust=90 betrayal=0.05. Engine must complete (no infinite loop), output valid JSON.

Fixture 'complete_rivalry': all pairs have trust=0.0 or betrayal_prob=0.99.

Expected: alliances = []. supporting_reps may be empty or minimal.

Fixture 'minimum_viable': exactly 1 rep, 1 proposal, no objections, no relationships.

Expected: engine completes, proposal selected, rep in supporting_reps.

■ TEST:

```
pytest tests/scenarios/test_adversarial.py -v # all 5 must pass
pytest tests/ -k 'hypothesis' -v # property tests
pytest tests/scenarios/ -v 2>&1 | grep -E 'PASSED|FAILED' | wc -l # expect 18
```

PHASE 5 · T+210 → T+240 MIN

FINAL INTEGRATION, SUBMISSION PREP

STEP 27 — P3: Full 18-Scenario Run + Coverage Gate — P3

- Run ALL 18 scenario tests: pytest tests/scenarios/ -v
- Count: must be 18 PASSED, 0 FAILED
- Run coverage gate: pytest tests/ --cov=src --cov-fail-under=90
- If coverage below 90%: identify uncovered modules, P3 adds targeted unit tests
- Run full pipeline on actual sample data: python consensus_engine.py
- Validate output/final_agreement.json against schema with Pydantic

■ TEST:

```
pytest tests/scenarios/ -v --tb=short 2>&1 | tail -30
pytest tests/ --cov=src --cov-report=term-missing --cov-fail-under=90
python -c "from src.output.formatter import ConsensusOutput; import json;
ConsensusOutput(**json.load(open('output/final_agreement.json'))); print('SCHEMA OK')"
```

STEP 28 — P2: Final Algorithm Review + Performance Check — P2

- Run: python consensus_engine.py --profile — check per-stage timing
- Verify score matrix build is <10ms at n=50 (numpy vectorized)
- Verify full pipeline is <500ms at n=50
- Final review: are thresholds calibrated (not hardcoded) in the actual pipeline?
- Final review: is Floyd-Warshall transitive risk actually called before trojan_filter?
- Final review: is combination enumeration used for proposals ≤ 30?

■ MANUAL STEP:

→ python consensus_engine.py --profile | head -20 → check no O(n^3) Python loops

→ grep -r 'TROJAN_BETRAYAL_THRESHOLD = 0.70' src/ → should return NOTHING (it's now data-driven)

→ grep -r 'compute_transitive_risk' src/ → should appear in the pipeline

STEP 29 — P1: Docs + Submission Packaging — P1

- Write APPROACH.md: 3-4 paragraphs explaining the 14-stage pipeline, key formulas, and design decisions
- Write LIMITATIONS.md: honest self-critique (directly from Section 12 of architecture doc)
- Write RESULTS.md: what the engine outputs on the sample data
- Verify README.md has: install instructions, how to run, how to run tests
- Tag final commit: git tag v1.0.0-hackathon && git push --tags
- Zip the repo for submission, verify zip extracts cleanly

■ ANTIGRAVITY PROMPT:

```
Write APPROACH.md for the Phantom Consensus engine. Structure: 1) Problem Analysis
(why naive sort-by-priority fails, list the 18 hidden test traps). 2) Pipeline Design
(14-stage DAG, why each stage exists). 3) Key Algorithms (relationship scoring formula,
alliance detection with NetworkX, cascading betrayal with Floyd-Warshall, data-driven
threshold calibration). 4) Engineering Choices (Pydantic v2 for type safety, structlog
for audit trail, NumPy for vectorized scoring, row-level fault isolation for CSV parsing).
Be specific and confident — this is a judging artifact.
```

■ MANUAL STEP:

→ make lint → must exit 0

→ make test → must exit 0 with 90%+ coverage

→ make run → must produce output/final_agreement.json

→ git tag v1.0.0-hackathon && git push --tags

STEP 30	ALL: Final Pre-Submission Checklist	ALL

- ■ python consensus_engine.py exits 0 and produces valid JSON
- ■ pytest tests/scenarios/ -v shows 18/18 PASSED
- ■ pytest tests/ --cov-fail-under=90 exits 0
- ■ ruff check src/ tests/ exits 0 (no lint errors)
- ■ mypy src/ --strict exits 0 (no type errors)
- ■ output/final_agreement.json passes schema validation
- ■ APPROACH.md, LIMITATIONS.md, RESULTS.md exist and are non-trivial
- ■ Threshold calibration is DATA-DRIVEN (not hardcoded 0.7)
- ■ Floyd-Warshall transitive risk is in the pipeline
- ■ Combination enumeration replaces greedy for p≤30
- ■ Quarantine log shows in structured logs (demonstrates dirty data handling)
- ■ --query CLI flag works: python consensus_engine.py --query rep_001
- ■ Dashboard: python dashboard/visualize.py runs without error
- ■ CI/CD: GitHub Actions workflow is green or at least committed

■ TEST:

```
make all # lint + test + run in one command
cat output/final_agreement.json | python -m json.tool | head -30
```

FINAL VERDICT	Execute this plan completely → 18/18 hidden tests → FIRST PLACE